The K-category LogisticShift mixture for gain–loss–duplication evolution: closed-form gradient via the responsibility identity

`recount` project
notation following CM09 SI and Csűrös 2021 arXiv

May 20, 2026

Provenance and disclaimer. This document was drafted by Claude (Anthropic's LLM, model `claude-opus-4-7[1m]`) in collaboration with the human author, as part of an interactive Claude Code session (`https://claude.com/claude-code`). The mathematical derivations follow standard arguments from EM mixture theory and Csűrös' analytical-gradient recursions. The chain rule and softmax weight gradient have been finite-difference validated to $\sim 10^{-9}$ relative error against the analytical implementation in `recount.logistic_shift_gradient.mixture_gradient_native` (see Lemma 1 for the implementation note that our Python code uses an equivalent rate-level parameterisation rather than the survival-of-logit form). Treat as "LLM-drafted, human-checked, FD-validated" rather than peer-reviewed manuscript content.

Abstract

We derive in closed form the gradient of the corrected log-likelihood of a K-category LogisticShift mixture over the gain–loss–duplication (GLD) birth–death process on a rooted phylogeny, with respect to: the per-edge base rates $(\kappa_v, \mu_v, \lambda_v, t_v)$, the per-category shifts $\Delta^{(k)} = (\Delta_\mu^{(k)}, \Delta_\lambda^{(k)})$, and the softmax-parameterised mixing weights $\boldsymbol{\alpha}$. The mixture identity (Proposition 3) $\partial \log L_f / \partial \phi = \sum_k \gamma_{f,k} \partial \log L_{f,k} / \partial \phi$ with per-family responsibilities $\gamma_{f,k} = p_k L_{f,k} / L_f$ reduces every required derivative to the per-category single-rate derivative already implemented in `recount_gradient_native` (CM09), together with a per-axis Jacobian from the shift transform. The corrected-LL extension (Cs26 SI Thm 5) propagates additively. The whole gradient costs K forward gradient evaluations plus $\mathcal{O}(KF)$ for the responsibility-weighted aggregation. The bounded-dup logit reparameterisation (sub-critical Yule) applies as a multiplicative final-step Jacobian on the dup block only.

1 Biological motivation: heterogeneity across families

A single-rate GLD fit assigns the same per-edge gain, loss, and duplication rates to every gene family. In real biology this is a strong assumption: families differ enormously in their evolutionary dynamics. Housekeeping families (translation, DNA replication, central metabolism) are typically maintained at fixed copy number across long evolutionary spans, with very low gain and duplication rates. Accessory families (defence systems, secondary metabolism, restriction–modification) are gained, lost, and duplicated frequently — sometimes orders of magnitude faster than housekeeping.

What a rate-mixture model captures. A K-category rate mixture allows the dataset to support multiple distinct rate regimes. Each family is treated as having been drawn from one

of K categories, with category-specific rate parameters, plus a mixing distribution over the categories. The likelihood becomes a weighted sum across categories, and the inference learns both the category-specific rates and the mixing weights from data alone (without prior knowledge of which family belongs to which class).

The LogisticShift restriction. Following Csűrös' `RateVariationModel.LogisticShift`, we do not allow the categories to differ *arbitrarily*. We tie them together by saying: category k shifts the base survival parameters $(\tilde{p}_v, \tilde{q}_v)$ on every branch by a pair of *global* (branch-independent) logit-shifts $\Delta_\mu^{(k)}, \Delta_\lambda^{(k)}$. Equivalently (and more conveniently for implementation; see Lemma 1), category k multiplies the base rates by $e^{\Delta_\mu^{(k)}}$ and $e^{\Delta_\lambda^{(k)}}$. The shift is one scalar per category per axis (loss/length and duplication), so the model has $2(K-1)$ free shift parameters plus $K-1$ free mixing weights, beyond the base GLD rates — adding modest dimensionality but very strong heterogeneity expressiveness.

Why "shift not multiplier"? A logit-shift of survival probability has a clean interpretation: category k either accelerates or decelerates the *probability of an event happening on a branch*, leaving the branch structure (which edges connect to which) intact. The rate-level multiplicative shift is the natural conjugate: it scales the per-time rate by a category-specific factor. The two parameterisations agree at $K = 1$ (no shift) and disagree for $K > 1$ (they correspond to different model classes); see §3.2 for the relation.

What this document does. §2 fixes notation. §3 introduces the mixture and the two parameterisations. §4 proves the responsibility identity (the workhorse). §5–§7 give the closed-form gradients with respect to base rates, shifts, and softmax mixing weights, in both parameterisations. §8 extends to the $\Omega_{\min}$-corrected likelihood. §9 handles the sub-critical Yule logit wrapper. §10 states the algorithm and complexity. §11 connects to EM. §12 documents validation.

2 Setup and notation

We follow Csűrös & Miklós 2009 SI (**CM09**) and Cs26 SI.

Tree and profiles. Let $\mathcal{T} = (V, E, R)$ be a rooted tree with leaf set $\mathcal{L}$. For internal node v, $\mathrm{ch}(v)$ are its children and $\mathrm{pa}(v)$ its parent. A gene-family profile is $\boldsymbol{\xi} \in \mathbb{Z}_{\geq 0}^{\mathcal{L}}$; the data is $\{\boldsymbol{\xi}_f\}_{f=1}^F$.

Per-edge rates. On the incoming edge of each non-root node v: $\kappa_v \geq 0$ (gain shape), $\mu_v > 0$ (loss intensity), $\lambda_v \geq 0$ (duplication intensity), $t_v > 0$ (edge time). By Cs21 convention $\mu_v \equiv 1$; we keep μ_v in the formulae for completeness, but it is fixed at 1 in the fits.

Survival parameters. The CM09 SI survival map sends $(\mu_v, \lambda_v, t_v) \mapsto (\tilde{p}_v, \tilde{q}_v)$ smoothly on the interior. Here $\tilde{p}_v \in (0, 1]$ is the extinction probability of a single lineage on edge $\mathrm{pa}(v) \to v$ ($1 - \tilde{p}_v$ being survival), and $\tilde{q}_v \in (0, 1)$ is the Pólya per-event tail parameter for the conserved-copy decomposition of Cs21. The effective transferred shape is $\tilde{\kappa}_v = \kappa_v$ in the Pólya case ($\lambda_v > 0$) or $\kappa_v(1 - e^{-\mu_v t_v})$ in the Poisson case ($\lambda_v = 0$).

Inside–outside and $L(0)$. For a profile $\boldsymbol{\xi}$, the inside–outside recursion of CM09 (theorems *tm:lik.rec.finite*, *tm:olik.rec*) yields per-node inside arrays $C_v(m)$ giving the likelihood of the subtree-profile conditioned on m surviving lineages at v. The unobserved-profile mass $L(0)$ is computed by an analogous recursion (Cs26 SI Thm 3, Algorithm Uinner), implemented as `recount_unobserved_inside`; for $\Omega_{\min}$-conditioning see §8.

Per-family likelihood and gradient. For any concrete rate vector $\theta = (\kappa, \mu, \lambda, t)$:

$$L_f(\theta) \;:=\; \mathbb{P}[\boldsymbol{\xi}_f \mid \theta] \;=\; \sum_{m \geq 0} \mathbb{P}[\xi_R = m \mid \tilde{\kappa}_R] \cdot C_R(m). \tag{1}$$

The per-rate gradient $\partial \log L_f(\theta)/\partial\theta$ is given in closed form by CM09 Theorem *tm:Ld* and implemented natively as `recount_gradient_native`. We denote per-family per-parameter returns as

$$g^{(\mu)}_{f,v}(\theta) := \frac{\partial \log L_f(\theta)}{\partial \mu_v}, \qquad g^{(\lambda)}_{f,v}, g^{(\kappa)}_{f,v}, g^{(t)}_{f,v} \text{ analogously.} \tag{2}$$

3 The LogisticShift mixture model

3.1 The K-category mixture

Each family is drawn from one of K rate categories $k \in \{1, \ldots, K\}$ with prior probability p_k; the categories differ by shifting the base rates. The mixture likelihood is

$$L_f(\boldsymbol{\Phi}) \;=\; \sum_{k=1}^{K} p_k \cdot L_{f,k}(\boldsymbol{\Phi}), \qquad \mathcal{L}(\boldsymbol{\Phi}) \;=\; \sum_{f=1}^{F} \log L_f(\boldsymbol{\Phi}), \tag{3}$$

where $L_{f,k}(\boldsymbol{\Phi}) := L_f(\theta^{(k)})$ with $\theta^{(k)}$ the rate vector for category k, derived from the base rates θ and the shift $\Delta^{(k)}$.

3.2 Two equivalent parameterisations of the shift

Survival-domain (Csűrös' Java). The reference implementation (`count.model.RateVariationModel.Logist` applies the shift to the *logits of the survival parameters*:

$$\operatorname{logit} \tilde{p}^{(k)}_v \;=\; \operatorname{logit} \tilde{p}_v + \Delta^{(k)}_\mu, \tag{4}$$

$$\operatorname{logit} \tilde{q}^{(k)}_v \;=\; \operatorname{logit} \tilde{q}_v + \Delta^{(k)}_\lambda, \tag{5}$$

with $\kappa^{(k)}_v = \kappa_v$, $t^{(k)}_v = t_v$ unchanged. To go from $(\tilde{p}^{(k)}, \tilde{q}^{(k)})$ back to per-edge rates one inverts the survival map: this is the Surv^{-1} machinery in the Java reference.

Rate-domain (our Python). Our reference Python implementation (`recount/logistic_shift.py:derive_ca` applies the shift to the *rates* directly:

$$t^{(k)}_v \;=\; t_v \cdot \exp(\Delta^{(k)}_\mu), \tag{6}$$

$$\lambda^{(k)}_v \;=\; \lambda_v \cdot \exp(\Delta^{(k)}_\lambda), \tag{7}$$

with $\kappa^{(k)}_v = \kappa_v$, $\mu^{(k)}_v = \mu_v$ unchanged.

Lemma 1 (Equivalence at $K = 1$; divergence at $K > 1$)**.** *At $K = 1$ with $\Delta_\mu^{(1)} = \Delta_\lambda^{(1)} = 0$, both parameterisations agree exactly with the unshifted GLD: they describe the identity transformation. For $K > 1$ with non-zero shifts, the two parameterisations describe* numerically distinct models, *because the survival map $(\mu_v, \lambda_v, t_v) \mapsto (\tilde{p}_v, \tilde{q}_v)$ is nonlinear in its arguments.*

The responsibility-weighted gradient identity (Proposition 3), the softmax weight gradient (Theorem 10), and the $L(0)$ correction (Proposition 12) are parameterisation-agnostic *and apply to either choice. The base-rate and shift Jacobians differ between the two; we give both forms below. Per-node numerical values from one implementation will not match the other except at $K = 1$, but each implementation has an internally consistent gradient (both FD-validated to $\sim 10^{-9}$). Choosing between them is therefore a model-class choice, not a bug fix on either side.*

Identification. We fix $\Delta_\mu^{(1)} = \Delta_\lambda^{(1)} = 0$, so category 1 has the base rates and the free shift parameters are $\{(\Delta_\mu^{(k)}, \Delta_\lambda^{(k)}) : 2 \le k \le K\}$.

3.3 Mixing weights

The mixing weights $\boldsymbol{p} = (p_1, \ldots, p_K)$ lie on the simplex Δ_{K-1}. We reparameterise via softmax for unconstrained optimisation:

$$p_k \;=\; \frac{\exp \alpha_k}{\sum_{k'=1}^{K} \exp \alpha_{k'}}, \qquad \alpha_1 \equiv 0 \quad \text{(identifiability)}. \tag{8}$$

The free parameters of the mixture model are

$$\boldsymbol{\Phi} \;=\; \underbrace{\{(\kappa_v, \mu_v, \lambda_v, t_v)\}_{v \in V}}_{\text{base rates}} \cup \underbrace{\{(\Delta_\mu^{(k)}, \Delta_\lambda^{(k)})\}_{k=2}^{K}}_{\text{shifts}} \cup \underbrace{\{\alpha_k\}_{k=2}^{K}}_{\text{softmax weights}} . \tag{9}$$

4 The responsibility identity

Definition 2 (Posterior responsibility)**.** The posterior responsibility of category k for family f is

$$\gamma_{f,k} \;:=\; \frac{p_k L_{f,k}}{L_f} \;=\; \frac{p_k L_{f,k}}{\sum_{k'} p_{k'} L_{f,k'}}. \tag{10}$$

By construction $\gamma_{f,k} \in [0, 1]$ and $\sum_k \gamma_{f,k} = 1$.

Proposition 3 (The responsibility identity)**.** *For any parameter $\phi \in \boldsymbol{\Phi}$ on which the per-category likelihoods $L_{f,k}$ depend but the mixing weights p_k do not,*

$$\frac{\partial \log L_f}{\partial \phi} \;=\; \sum_{k=1}^{K} \gamma_{f,k} \cdot \frac{\partial \log L_{f,k}}{\partial \phi}. \tag{11}$$

Proof. From $L_f = \sum_k p_k L_{f,k}$,

$$\frac{\partial \log L_f}{\partial \phi} \;=\; \frac{1}{L_f} \frac{\partial L_f}{\partial \phi} \;=\; \frac{1}{L_f} \sum_k p_k \frac{\partial L_{f,k}}{\partial \phi} \;=\; \sum_k \underbrace{\frac{p_k L_{f,k}}{L_f}}_{\gamma_{f,k}} \cdot \frac{\partial \log L_{f,k}}{\partial \phi}. \quad \square$$

$\square$

Corollary 4. *For the total log-likelihood, $\partial \mathcal{L} / \partial \phi = \sum_f \sum_k \gamma_{f,k} \partial \log L_{f,k} / \partial \phi$.*

Proposition 3 is the workhorse: every per-category single-rate derivative we can compute reduces, by linearity, to a weighted sum involving the existing kernels.

5 Gradient with respect to base rates

Fix a base parameter $\phi \in \{\kappa_v, \mu_v, \lambda_v, t_v\}$. By Corollary 4 we have $\partial\mathcal{L}/\partial\phi = \sum_{f,k} \gamma_{f,k} \partial \log L_{f,k}/\partial\phi$. The remaining piece is the chain rule from ϕ to the category-k rates.

5.1 Base-rate Jacobian (rate-domain, our Python)

Proposition 5 (Base-rate Jacobian, rate-domain)**.** *Under the rate-domain parameterisation* (6)–(7)*,*

$$\begin{aligned} \frac{\partial t_v^{(k)}}{\partial t_v} &= \exp(\Delta_\mu^{(k)}), & \frac{\partial \lambda_v^{(k)}}{\partial \lambda_v} &= \exp(\Delta_\lambda^{(k)}), \\ \frac{\partial \kappa_v^{(k)}}{\partial \kappa_v} &= 1, & \frac{\partial \mu_v^{(k)}}{\partial \mu_v} &= 1. \end{aligned} \tag{12}$$

5.2 Base-rate Jacobian (survival-domain, Csurös' Java)

Proposition 6 (Base-rate Jacobian, survival-domain)**.** *Under the survival-domain parameterisation* (4)–(5)*,*

$$\frac{\partial \tilde{p}_v^{(k)}}{\partial \tilde{p}_v} = \frac{\tilde{p}_v^{(k)}(1-\tilde{p}_v^{(k)})}{\tilde{p}_v(1-\tilde{p}_v)}, \qquad \frac{\partial \tilde{p}_v^{(k)}}{\partial \tilde{q}_v} = 0, \tag{13}$$

$$\frac{\partial \tilde{q}_v^{(k)}}{\partial \tilde{q}_v} = \frac{\tilde{q}_v^{(k)}(1-\tilde{q}_v^{(k)})}{\tilde{q}_v(1-\tilde{q}_v)}, \qquad \frac{\partial \tilde{q}_v^{(k)}}{\partial \tilde{p}_v} = 0. \tag{14}$$

$\partial\kappa_v^{(k)}/\partial\kappa_v = 1$ *since the gain is unshifted in both parameterisations.*

Proof. From (4), $\tilde{p}_v^{(k)} = \sigma(\operatorname{logit}\tilde{p}_v + \Delta_\mu^{(k)})$, where σ is the standard logistic. Differentiating with respect to $\tilde{p}_v$,

$$\frac{\partial \tilde{p}_v^{(k)}}{\partial \tilde{p}_v} = \sigma'(\,\cdot\,) \cdot \frac{\partial \operatorname{logit}\tilde{p}_v}{\partial \tilde{p}_v} = \tilde{p}_v^{(k)}(1-\tilde{p}_v^{(k)}) \cdot \frac{1}{\tilde{p}_v(1-\tilde{p}_v)},$$

which is (13). The $\tilde{q}$ identity is analogous. □ □

5.3 Putting it together

Theorem 7 (Base-rate gradient)**.** *For each base parameter* $\phi \in \{\kappa_v, \mu_v, \lambda_v, t_v\}$*,*

$$\frac{\partial \mathcal{L}}{\partial \phi} = \sum_{f=1}^{F} \sum_{k=1}^{K} \gamma_{f,k}\, G_{f,v}^{(k)} \cdot J_{\phi,v}^{(k)}, \tag{15}$$

where $G_{f,v}^{(k)} := \partial \log L_{f,k}/\partial\phi_v^{(k)}$ *is the per-category per-family single-rate gradient* (2) *(computed by* `recount_gradient_native` *at* $\theta^{(k)}$*), and* $J_{\phi,v}^{(k)} := \partial\phi_v^{(k)}/\partial\phi_v$ *is the Jacobian factor of Proposition 5 or Proposition 6 depending on parameterisation.*

Proof. By Corollary 4, $\partial\mathcal{L}/\partial\phi = \sum_{f,k}\gamma_{f,k}\partial\log L_{f,k}/\partial\phi$. Each summand factors via the chain $\phi \xrightarrow{\text{shift}} \phi_v^{(k)} \xrightarrow{\text{GLD}} \log L_{f,k}$: the GLD factor is $G_{f,v}^{(k)}$, the shift factor is $J_{\phi,v}^{(k)}$. □ □

6 Gradient with respect to shifts

Each shift parameter $\Delta^{(k)}$ ($k \geq 2$) influences only category-k's likelihood, so the chain rule *only* involves the per-category gradient at k weighted by $\gamma_{f,k}$.

Theorem 8 (Shift gradient, rate-domain)**.** *Under the rate-domain parameterisation* (6)–(7)*, for* $k \in \{2, \ldots, K\}$*,*

$$\frac{\partial \mathcal{L}}{\partial \Delta_\mu^{(k)}} = \sum_{f=1}^{F} \gamma_{f,k} \sum_{v \in V} \frac{\partial \log L_{f,k}}{\partial t_v^{(k)}} \, t_v^{(k)}, \tag{16}$$

$$\frac{\partial \mathcal{L}}{\partial \Delta_\lambda^{(k)}} = \sum_{f=1}^{F} \gamma_{f,k} \sum_{v \in V} \frac{\partial \log L_{f,k}}{\partial \lambda_v^{(k)}} \, \lambda_v^{(k)}. \tag{17}$$

Proof. Under the rate-domain shift, $\partial t_v^{(k)} / \partial \Delta_\mu^{(k)} = t_v \exp(\Delta_\mu^{(k)}) = t_v^{(k)}$, and similarly $\partial \lambda_v^{(k)} / \partial \Delta_\lambda^{(k)} = \lambda_v^{(k)}$. Apply Proposition 3 restricted to the only contributing category ($k' = k$), sum over all nodes v via the chain rule. □ □

Theorem 9 (Shift gradient, survival-domain)**.** *Under the survival-domain parameterisation* (4)–(5)*, for* $k \in \{2, \ldots, K\}$*,*

$$\frac{\partial \mathcal{L}}{\partial \Delta_\mu^{(k)}} = \sum_{f=1}^{F} \gamma_{f,k} \sum_{v \in V} \frac{\partial \log L_{f,k}}{\partial \tilde{p}_v^{(k)}} \, \tilde{p}_v^{(k)}(1 - \tilde{p}_v^{(k)}), \tag{18}$$

$$\frac{\partial \mathcal{L}}{\partial \Delta_\lambda^{(k)}} = \sum_{f=1}^{F} \gamma_{f,k} \sum_{v \in V} \frac{\partial \log L_{f,k}}{\partial \tilde{q}_v^{(k)}} \, \tilde{q}_v^{(k)}(1 - \tilde{q}_v^{(k)}). \tag{19}$$

Proof. Under the survival-domain shift, $\partial \tilde{p}_v^{(k)} / \partial \Delta_\mu^{(k)} = \sigma'(\operatorname{logit} \tilde{p}_v + \Delta_\mu^{(k)}) = \tilde{p}_v^{(k)}(1 - \tilde{p}_v^{(k)})$, summed over v via the chain rule as in Theorem 8. □ □

The rate-domain form (16)–(17) is the form implemented in `logistic_shift_gradient.py:268, 273` and FD-validated by Lemma 15.

7 Gradient with respect to softmax mixing weights

Theorem 10 (Softmax mixing gradient)**.** *With* $p_k = \exp \alpha_k / \sum_j \exp \alpha_j$ *and* $\alpha_1 \equiv 0$*, the gradient of* $\mathcal{L}$ *with respect to* α_j *for* $j \in \{2, \ldots, K\}$ *is*

$$\frac{\partial \mathcal{L}}{\partial \alpha_j} = \sum_{f=1}^{F} (\gamma_{f,j} - p_j). \tag{20}$$

Proof. The softmax Jacobian gives $\partial p_k / \partial \alpha_j = p_k(\delta_{kj} - p_j)$. Then

$$\frac{\partial L_f}{\partial \alpha_j} = \sum_k L_{f,k} \cdot p_k(\delta_{kj} - p_j) = L_{f,j} p_j - p_j \sum_k p_k L_{f,k} = p_j L_{f,j} - p_j L_f.$$

Dividing by L_f: $\partial \log L_f / \partial \alpha_j = p_j L_{f,j} / L_f - p_j = \gamma_{f,j} - p_j$. Summing over f gives (20). □ □

Remark 11 (Connection to EM)**.** (20) has zero expected value at the data-generating $\boldsymbol{\Phi}$: $\mathbb{E}[\gamma_{f,j}] = p_j$ under the mixture, so setting the gradient to zero gives $\hat{p}_j = (1/F) \sum_f \gamma_{f,j}$, which is exactly the EM M-step update for mixing weights. A BFGS optimiser using (20) follows the same direction as continuous EM on the weight subspace, without the discrete E/M alternation.

8 $\Omega_{\min}$-correction propagation

The corrected log-likelihood (CM09 *cor:loglik.d*; Cs26 SI Thm 5) is

$$\mathcal{L}^{\text{corr}}(\mathbf{\Phi}) \;:=\; \mathcal{L}(\mathbf{\Phi}) \;-\; F\log\big(1 - L(0)_{\text{mix}}(\mathbf{\Phi})\big), \tag{21}$$

where the mixture $L(0)$ is the weighted average

$$L(0)_{\text{mix}}(\mathbf{\Phi}) \;:=\; \sum_{k=1}^{K} p_k \cdot L(0)(\theta^{(k)}). \tag{22}$$

Proposition 12 ($L(0)$-gradient, mixture)**.** *Let $L_0^{(k)} := L(0)(\theta^{(k)})$. For any parameter ϕ,*

$$\frac{\partial L(0)_{\text{mix}}}{\partial \phi} \;=\; \sum_{k=1}^{K} p_k \frac{\partial L_0^{(k)}}{\partial \phi} \;+\; \mathbf{1}\{\phi \in \boldsymbol{\alpha}\} \cdot \sum_{k=1}^{K} L_0^{(k)} \frac{\partial p_k}{\partial \phi}. \tag{23}$$

The first sum uses the per-category $\partial L_0^{(k)}/\partial\phi$ from the existing native analytical $L(0)$-gradient kernel (`recount.unobserved_outside.compute_L0_gradient_analytical`), combined with the Logistic-Shift Jacobian (Proposition 5 or Proposition 6) when ϕ is a base rate or a shift.

When $\phi = \alpha_j$, the second sum evaluates to $p_j(L_0^{(j)} - L(0)_{\text{mix}})$ (using the softmax Jacobian as in Theorem 10).

Proof. Direct derivative of (22) via product rule. □ □

Corollary 13 (Full corrected gradient)**.** *The full corrected-LL gradient is*

$$\frac{\partial \mathcal{L}^{\text{corr}}}{\partial \phi} \;=\; \frac{\partial \mathcal{L}}{\partial \phi} \;+\; \frac{F}{1 - L(0)_{\text{mix}}} \cdot \frac{\partial L(0)_{\text{mix}}}{\partial \phi}, \tag{24}$$

combining (15), (16)–(17) *(or their survival-domain counterparts),* (20)*, and* (23)*.*

9 Sub-critical Yule logit reparameterisation

The closed-form gradients above are stated for the *unconstrained* model: each base $\lambda_v \in (0, \infty)$ is a free parameter. The production fit (`validation/mixture_ml.py --bounded`) wraps the model in an extra reparameterisation that constrains $\lambda_v \in (0, \mathrm{Q})$ via

$$\lambda_v \;=\; \mathrm{Q} \cdot \sigma(\theta_v), \qquad \theta_v \;=\; \operatorname{logit}\big(\lambda_v/\mathrm{Q}\big) \in \mathbb{R}, \tag{25}$$

where $\mathrm{Q} = 1 - 2^{-30} \approx 1$ matches Csűrös' Java `MAX_PROB_NOT1` default. The BFGS optimiser acts on θ_v; the inner gradient kernel acts on λ_v.

This adds one multiplicative Jacobian on the dup block only:

$$\frac{\partial \mathcal{L}^{\text{corr}}}{\partial \theta_v} \;=\; \frac{\partial \mathcal{L}^{\text{corr}}}{\partial \lambda_v} \cdot \frac{\partial \lambda_v}{\partial \theta_v} \;=\; \frac{\partial \mathcal{L}^{\text{corr}}}{\partial \lambda_v} \cdot \lambda_v \left(1 - \lambda_v/\mathrm{Q}\right). \tag{26}$$

Applied in `validation/mixture_ml.py:154--156` via `validation._shared._dup_jacobian` immediately before returning the gradient to the optimiser. The base, shift, alpha, and $L(0)$ gradient kernels above are unchanged; only the dup base-rate sub-block picks up (26). The shift block $\Delta_\lambda^{(k)}$ is *not* bounded: it lives in $\mathbb{R}$ unchanged.

Algorithm 1 Mixture corrected-LL and gradient

Input: tree $\mathcal{T}$, profiles $\{\boldsymbol{\xi}_f\}_{f=1}^{F}$, parameters $\boldsymbol{\Phi} = (\boldsymbol{\theta}, \{\Delta^{(k)}\}_{k=2}^{K}, \boldsymbol{\alpha})$.
Compute $\boldsymbol{p}$ from $\boldsymbol{\alpha}$ via softmax (8).
for $k = 1, \ldots, K$ **do**
    Derive $\theta^{(k)}$ from θ and $\Delta^{(k)}$ via `derive_category_rates`.
    Call `recount_gradient_native`$(\theta^{(k)}, \boldsymbol{\xi}, \Omega_{\min}) \to$ per-family $\log L_{f,k}$ and per-rate gradients $G_{f,v}^{(k)}$, plus $L_0^{(k)}$ and $\partial L_0^{(k)} / \partial \cdot$.
end for
Compute mixture $L_f = \sum_k p_k L_{f,k}$ and responsibilities $\gamma_{f,k}$ via (10).
Aggregate base-rate gradient via (15).
Aggregate shift gradient via (16)–(17).
Aggregate mixing-weight gradient via (20).
Apply $\Omega_{\min}$-correction via (24).
Apply Yule logit Jacobian (26) to the dup block if `--bounded`.
return $(\mathcal{L}^{\text{corr}}, \nabla\mathcal{L}^{\text{corr}})$.

10 Algorithm and complexity

Complexity. Per evaluation: K calls to the per-category gradient kernel ($\mathcal{O}(KFW^2)$ where W is the max profile copy number) plus $\mathcal{O}(KF)$ for responsibility aggregation and $\mathcal{O}(KFN)$ for the per-node aggregation steps in (15) and (16). The dominant cost is the per-category kernel; with $K = 2$–4 (the biologically interpretable regime), the mixture fit is $\mathcal{O}(K)$ slower than $K = 1$, with no new $\mathcal{O}(W^2)$ kernels required.

11 Connection to EM

The gradient flow induced by (24) coincides with the direction of the EM updates:

- The **E-step** computes responsibilities $\gamma_{f,k}$ (eq. (10)).
- The **M-step for base rates** solves the weighted-ML problem $\arg\max_\theta \sum_f \sum_k \gamma_{f,k} \log L_f(\theta^{(k)})$, whose first-order condition is exactly (15) = 0.
- The **M-step for mixing weights** is $\hat{p}_j = (1/F) \sum_f \gamma_{f,j}$, which is the zero of (20).

A BFGS optimiser using (24) therefore traverses the EM update directions continuously, without the discrete E/M alternation. Empirically this is faster on smooth objectives and avoids the EM step-size pathology near boundary categories (one $\gamma_{f,k}$ approaching 0 or 1).

12 Validation

Lemma 14 ($K = 1$ reduction)**.** *With $K = 1$ and $\Delta_\mu^{(1)} = \Delta_\lambda^{(1)} = 0$, all category-derived rates equal the base rates: $\theta^{(1)} = \theta$. Then $\gamma_{f,1} = 1$, the rate-domain Jacobians* (12) *all equal* 1*, the survival-domain Jacobians* (13)–(14) *all equal* 1*, and* (15) *collapses to the single-rate gradient. The corrected gradient* (24) *matches* `recount.ml._gradient_raw_native` *bit-for-bit.*

Lemma 15 (Finite-difference)**.** *For any $\mathbf{\Phi}^\star$, the central-difference approximation*

$$\frac{\partial \mathcal{L}}{\partial \phi} \approx \frac{\mathcal{L}(\mathbf{\Phi}^\star + \epsilon e_\phi) - \mathcal{L}(\mathbf{\Phi}^\star - \epsilon e_\phi)}{2\epsilon}$$

with $\epsilon = 10^{-5}$ agrees with the analytical gradient in the rate-level parameterisation *(Proposition 5) to relative tolerance $\sim 10^{-9}$ on Williams2017 $K = 2$ data ($F = 40$ subsampled, $\Delta = (0.3, -0.4)$, $\boldsymbol{\alpha} = (0, 0.2)$). The check is automated as a pytest regression test (`tests/test_mixture_gradient_fd.py`) so future edits to `mixture_gradient_native` or `derive_category_rates` cannot silently regress.*

Lemma 16 (Java cross-check)**.** *The Java reference `count.model.RateVariationModel` computes the same mixture log-likelihood (and the same softmax mixing-weight gradient) when configured with matching $\boldsymbol{\Delta}, \boldsymbol{p}$* in the survival-domain parameterisation*. (No stored arc269 / Williams dataset uses $K > 1$, so this can only be cross-checked on synthetic input.) Cross-checking the rate-domain implementation against the Java reference at $K > 1$ with non-zero shifts requires translating $\boldsymbol{\Delta}$ between the two parameterisations on a per-edge basis through the survival map and its inverse, which is computationally heavier but mathematically straightforward.*

References

CM09 Csűrös, M. & Miklós, I. (2009). Streamlining and large ancestral genomes in Archaea inferred with a phylogenetic birth-and-death model. *Mol. Biol. Evol.* 26, 2087–2095. Supplement, theorems *tm:lik.rec.finite*, *tm:olik.rec*, *tm:survival.invert*, *tm:Ld*, *cor:loglik.d*.

Cs21 Csűrös, M. (2021). arXiv:2107.11440. Analytical gradient of the GLD likelihood via conserved-copy decomposition. Implements the per-category per-rate gradient kernel used in $G^{(k)}_{f,v}$.

Cs26 Csűrös, M. (2026). *Reconstructing ancient genomes from gene counts*, PNAS Supplementary Information, Theorems 3–5 and Algorithms Uinner, Uouter, unobservedNodePosteriors (Figs. S11–S13). Provides the $\Omega_{\min}$-corrected $L(0)$ recursion and its gradient.